Module 2 – Introduction to Programming

(Theory Exercise)

1. **Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.**

Ans: C was developed by **Dennis Ritchie** at Bell Labs in the early 1970s. It evolved from the B language and was used to rewrite the Unix operating system.

- **Importance:** It is a "middle-level" language, combining the speed of low-level assembly with the ease of high-level languages.
- **Today:** It remains vital because it is fast, portable, and allows direct memory access, making it the backbone of modern computing.

2. **Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.**

Ans: Steps to Install:

1. **Compiler:** Download **MinGW** (which includes the GCC compiler).
2. **Environment Path:** Add the MinGW `bin` folder to your System Environment Variables.
3. **IDE:** Install **VS Code** or **Dev-C++**.
4. **Extension:** In VS Code, install the "C/C++" extension by Microsoft.

3. **Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.**

Ans: Key Components

- **Header:** `#include <stdio.h>` (Input/Output library).

- **Main Function:** `int main()` is the entry point where execution starts.
- **Variables:** Containers for data (e.g., `int age = 18;`).
- **Comments:** `//` for single line, `/* */` for multi-line.

4. Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

Ans: Operator Types:

- **Arithmetic:** `+, -, *, /, %` (Math)
- **Relational:** `==, !=, >, <` (Comparison)
- **Logical:** `&&` (AND), `||` (OR), `!` (NOT)
- **Assignment:** `=` (Store value)

5. Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

Ans: Decision Making:

- **if-else:** Executes a block of code if a condition is true.
- **Switch:** Selects one of many code blocks to be executed based on a variable's value.

6. Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

Ans: Comparison:

- **for:** Best when you know exactly how many times to repeat.
- **while:** Best when the loop depends on a condition being met.

- **do-while:** Guaranteed to run at least once.

7. Explain the use of break, continue, and goto statements in C. Provide examples of each.

Ans: break, continue, goto:

- **break:** Exits the loop entirely.
- **continue:** Skips the current iteration and goes to the next.
- **goto:** Jumps to a specific labeled part of the program (rarely used).

8. What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

Ans: Three Parts:

1. **Declaration:** Telling the compiler about the function.
2. **Definition:** Writing the actual code for the function.
3. **Call:** Running the function in `main()`.

9. Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

Ans: 1D vs 2D:

- **1D Array:** A single list of items (e.g., `int marks[5]`).
- **2D Array:** A table with rows and columns (e.g., `int matrix[3][3]`).

10. Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

Ans: Definition:

A **pointer** is a variable that stores the memory address of another variable. They are essential for dynamic memory allocation and efficient array handling.

11. Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr(). Provide examples of when these functions are useful.

Ans: Handling Functions:

- `strlen()`: Finds length.
- `strcpy()`: Copies string.
- `strcat()`: Joins (concatenates) two strings.
- `strcmp()`: Compares two strings.

12. Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

Ans: Concept:

A **Structure** (`struct`) allows you to group different data types (like an `int` and a `char`) into a single unit.

13. Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

Ans: Operations:

1. `fopen()`: Opens a file (modes: "w" for write, "r" for read).
2. `fprintf()` / `fscanf()`: Writes to or reads from the file.
3. `fclose()`: Closes the file to save changes.